Self-describing GOAL traces for LogGOPSim

DPHPC 2026 progress presentation

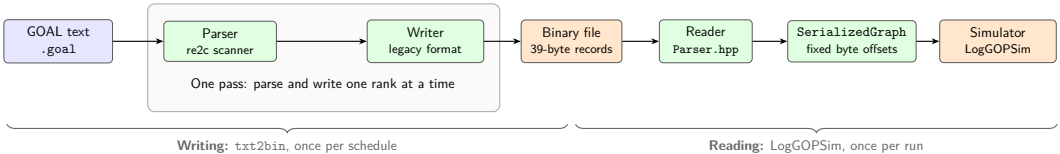
Joshua Durrant, Alexander Falter, Samuel Garcin, Mykhaylo Gershman

Supervised by Timo Schneider

17.11.26

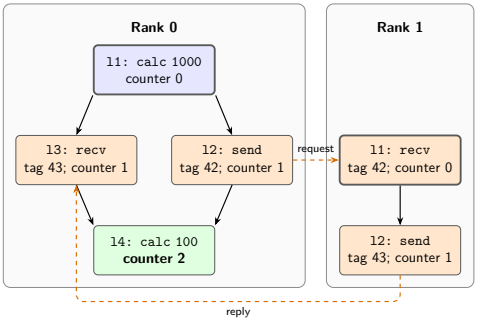
LogGOPSim today

LogGOPSim predicts how long an MPI program runs. Input: per rank, a graph of sends, receives and computations. Output: the predicted runtime.



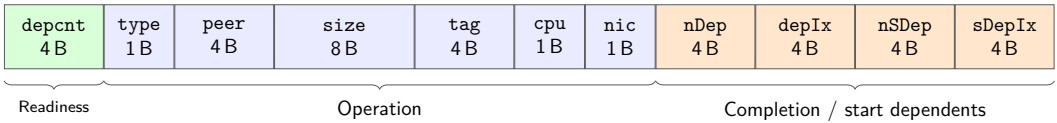
- `txt2bin` converts the text schedule once; each simulation then reads the binary file.
- The reader is built into LogGOPSim and knows the record layout by heart.

A schedule is a dependency graph



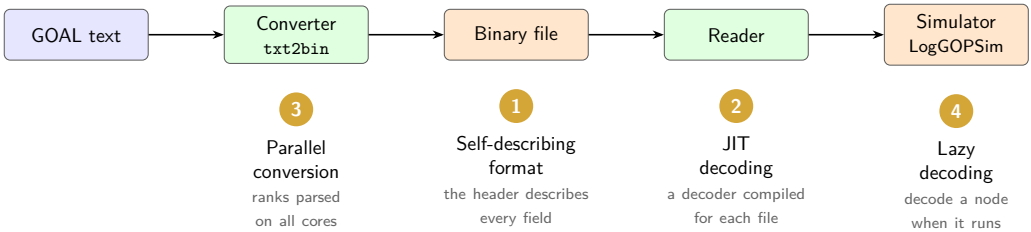
- Each rank is a graph of `calc`, `send` and `recv` nodes.
- A node's **counter** says how many nodes it still waits for. Roots start at 0.
- The simulator runs ready nodes with LogGOPS costs (L, o, g, G), then decrements the counters of their dependents.
- A dependent whose counter reaches 0 becomes ready.
- Output: the time at which each rank finishes.

The problem: a fixed 39-byte record



- **Wasteful:** every field at full width, whatever values occur. Most traces need 2–4 bytes per node.
- **Closed:** no room for extra information. ATLAHS hides the GPU in `nic` and packs five numbers into the tag; a wrong guess gives wrong results silently.
- **Hard-wired:** each simulator needs its own copy of the byte offsets.

Our design in four steps



Step 1 solves the problem. Steps 2–4 make sure the new format costs nothing: reading, converting and simulating get faster, not slower.

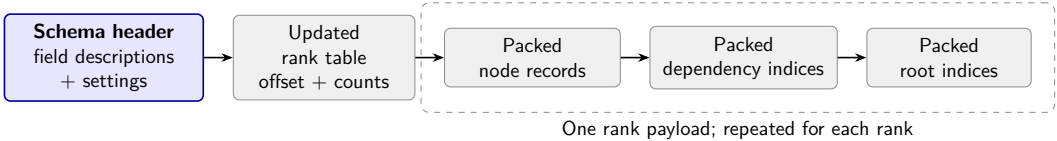
Step 1

Self-describing format

The file says what is in it

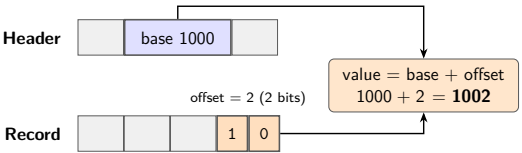


The file describes itself



- The **schema header** lists every field: name, type, bit position, width, base. A reader needs no built-in knowledge of the layout.
- All records of a file share one layout, so node i of a rank is at `rank_start + i` $\times$ `record_size`, without scanning.
- A new magic number tells the formats apart: legacy files stay readable through the same reader.

Each field gets only the bits it needs



- Width from the field's largest value.
- **Offset encoding:** the header stores the smallest value (the base), records store the offset.
- A constant field takes **0 bits**: its value lives only in the header.
- bcast_64k: 39 $\rightarrow$ 4 bytes per node.

In the grand picture: the layout depends on every value in the trace. The converter must see all values before writing (Step 3), and the reader must interpret the layout (Step 2).

Custom fields: the ATLAHS case

```
before: l2: send 1024b to 1 tag 3050012001 cpu 0 nic 2
after:  l2: send 1024b to 1 tag 0 cpu 0 nic 0 stream_id 0 gpu_id 2 peer_gpu_id 2
            chunk_id 3 channel_id 5 sequence_id 12 message_kind 0 communicator_id 1
```

- Custom fields are declared in TOML and treated like standard ones.
- Each simulator binds only the fields it needs; a missing field or a value too large for its member fails at load time.
- Explicit fields exposed and fixed two htsim bugs: messages sent to the wrong GPU, and different messages sharing a tag.

Recorded Llama run, 5.7M operations:

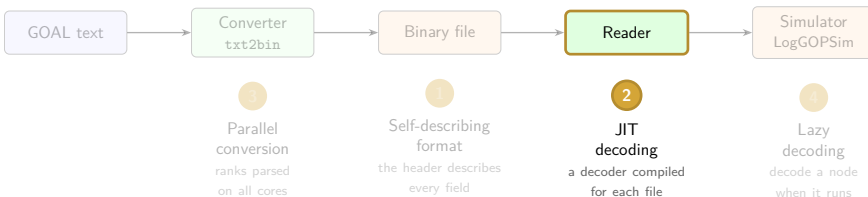
54% smaller

with all eight extra fields

Step 2

JIT decoding

A flexible format must not be slower to read



Flexibility has a cost at read time

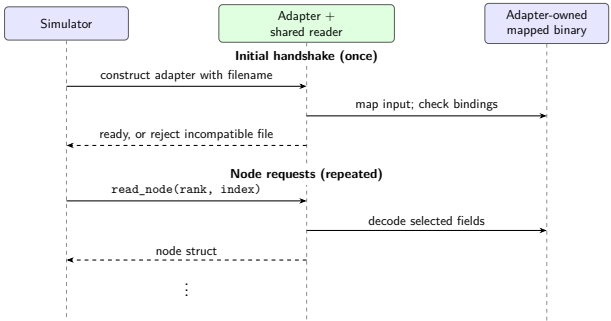
40%
bcast_64k

47%
dissem_16k

2%
alltoall_256
message matching dominates

- Share of the simulation time spent decoding, with the first, generic decoder.
- Bit positions are known only when the file is opened, so it interpreted the header for every field of every node: look up position, shift, mask, add base, check range.

Build the decoder once, run it for every node



- **Once, when the file is opened:** take the fields the simulator binds, build a **decode plan** (one step per field: offset, shift, mask, base, destination), pick a runner.
- **For every node:** the runner turns the record's bits into the simulator's node struct.

Compiled runners hard-code the layout

```
int decode(const u8* r, u8* const* t) {
  u64 w, c;
  // peer: 16 bits at bit 3
  memcpy(&w, r + 0x0, 8);
  c = (w >> 0x3) & 0xffff;
  store32(t[0] + 0x3c, c);
  // size: 0 bits, constant 1024
  store64(t[0] + 0x40, 0x400);
  // ... further fields
  return 1;
}
```

Generated C for bcast_64k, abridged.

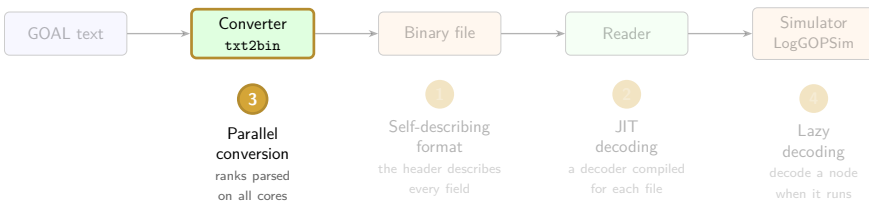
- Every position and mask is a constant: no loop, no lookup.
- `jit`: compiled with `cc`, cached on disk (about 340 ms the first time).
- `orc`: the same function in memory with LLVM (about 2.5 ms).
- `plan`: interpreter, the default and fallback.
- Only numbers from the header go into the code, so a trace can't inject code.

`jit`: 12–17 ns per node, 2× the interpreter, within 6% of code compiled ahead of time.

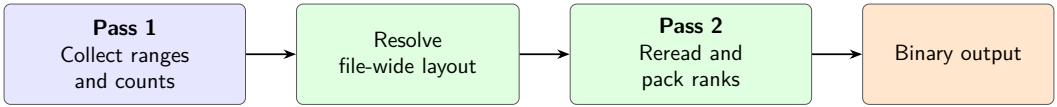
Step 3

Parallel conversion

Converting is now two passes; make them use all cores



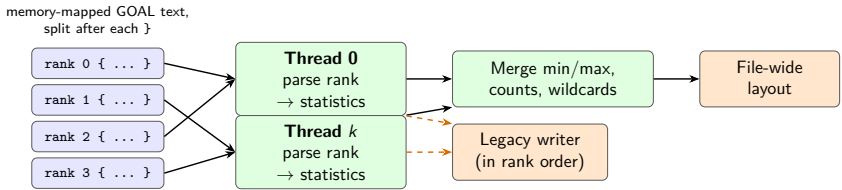
Why conversion reads the input twice



- The layout must be fixed before the first record is written: if tags 0–7 fit in 3 bits but a later rank uses tag 1000, earlier records can't be widened.
- **Pass 1** only collects statistics: each field's minimum and maximum, wildcards, counts.
- **Pass 2** parses again and packs each rank with the final layout.
- One rank's graph in memory at a time, so memory stays bounded.

Pass 1 was one sequential scan through a `FILE*`: 22–38 s on our largest traces.

Pass 1 in parallel: one rank per task



- In GOAL, ranks are independent blocks. The memory-mapped text is cut after each rank's closing brace (skipping comments).
- OpenMP threads parse whole ranks with the unchanged scanner and keep their own statistics.
- Minimum, maximum and counts merge in any order, so the result doesn't depend on the thread count. `txt2bin --threads N`.

Same result as before, and what it took

Identical to the sequential pass

- Same output bytes in both formats.
- Same errors: the lowest invalid rank is reported, with its line number in the whole file.
- 9 differential tests at 1, 2, 3 and 8 threads.
- Pipes can't be mapped: they use the sequential reader.

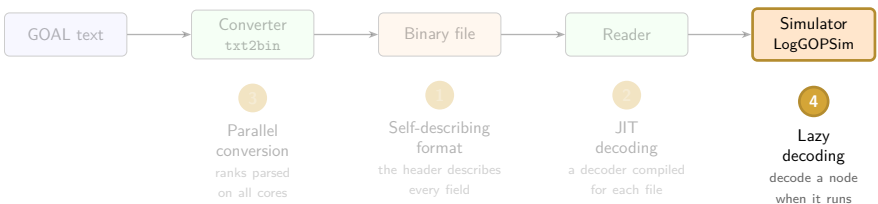
Two bottlenecks found on the way

- The legacy writer remapped the whole output file for every rank. With threads, each unmap got expensive: 45 s instead of 4 s. Now each rank is appended with `pwrite`.
- Threads waiting in `omp ordered` spin and starved the writer: 15 s instead of under 1 s. Now they sleep on a condition variable.

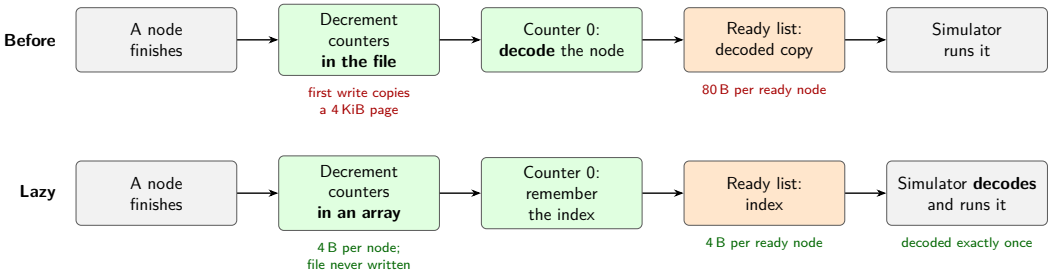
Step 4

Lazy decoding

Decode a node only when the simulator runs it



Where the simulator spends memory



- **Before:** counters are decremented in the memory-mapped file. The mapping is private, so the file stays reusable, but every page written is copied: a long run copies most of the file.
- **Lazy:** counters move to a small array, ready nodes are indices, and each node is decoded once, when it runs.

Implementation

```
// A node finished: release its dependents.
for (auto i : dependents) {
  auto& c = counters_[i];  // remaining + 1
  if (c == 0)              // first use
    c = reader.read_dependency_count(rank, i) + 1;
  if (--c == 1)            // none left
    ready_.push_back(i);   // 4-byte index
}

// The simulator takes the ready nodes: decode now.
for (auto i : ready_)
  out.push_back(reader.read_node(rank, i));
```

From `SimulationSchedule.hpp`, simplified.

- Counters are read from the file only on their first decrement: no extra pass over the file at startup.
- 0 means “not read yet”, so the stored value is the remaining count plus one.
- Works for both formats and every decoder, through the same reader.
- Opt-in flag `--lazy-decoding`; the default is unchanged.

Same simulation, fewer copies

Correctness

- The full `-v` event log is identical in both modes: every operation at the same simulated time.
- Checked on dissemination, ResNet and ring traces (14.7M–72.8M log lines), both formats.
- The trace file is never written, so it can be shared by concurrent runs.

Trade-offs

- The counter array is 4 B per node, allocated at startup: 1 billion nodes need 4 GB, still far less than the page copies it replaces.
- Not combinable with `--save-mem`, which exists only to write counters back to the file.

Files are 53–93% smaller

Trace	Bytes per node	File size	Smaller
bcast_64k	39 → 4	6.9 → 1.9 MiB	73%
dissem_16k	39 → 4	19.2 → 2.4 MiB	88%
alltoall_256	39 → 2	5.4 MiB → 404 KiB	93%
wide_64k (random 64-bit values)	39 → 15	6.9 → 3.2 MiB	53%
ATLAHS Llama run (+8 fields)	39 → 17	240.6 → 111.4 MiB	54%

Narrow and constant fields shrink most; even random 64-bit values save half.

Simulations are faster than the original

Trace	Original	Ours, legacy file	Ours, new + `jit`	`jit` vs. original
bcast_64k	48.4	43.9	43.7	−10%
dissem_16k	146.4	122.7	110.8	−24%
ring_256	39.8	32.8	32.9	−17%
alltoall_256	1950	2004	1996	+2%
wide_64k	crashes	78.4	79.5	

- Complete LogGOPSim runs in ms, identical results. alltoall is dominated by message matching.
- Decoding alone: `jit` is about 2× faster than the interpreter and within 6% of code compiled ahead of time.

Apple M3 Pro, median of 21 interleaved runs. Original: last upstream commit (76451de).

Parallel conversion and lazy decoding

Converter pass 1, seconds

Trace	Seq.	8 threads	
dissem_65536	2.79	0.60	**4.6**×
ring_2048	22.1	8.6	**2.6**×
alltoall_4096	38.1	10.7	**3.6**×

Peak memory of a simulation, MB

Trace	Before	Lazy	
dissem_524288	2220	1540	−31%
ring_2048	1647	1103	−33%
resnet_8192	216	153	−29%

- Two threads nearly halve pass 1; beyond four it flattens. Pass 2 is still sequential.
- Lazy decoding is also 2–11% faster.

WSL2 with 8 vCPUs, median of 3 runs. Lazy: new format, `plan` decoder; peak RSS.

Outlook

Next optimisations

- Parallelise pass 2 and the legacy writer.
- Dictionary encoding for few, far-apart values.
- Choose the decoder automatically from trace size and startup cost.
- Full benchmark study on Euler.

Conversion on the GPU (CUDA)

- Ranks and nodes are independent: one GPU thread per node instead of one CPU thread per rank.
- Field statistics are min/max reductions; packing records is independent per node.
- GPU memory bandwidth for streaming very large traces.

Who did what

Name	Work
Joshua Durrant	Full benchmarking study (upcoming)
Alexander Falter	Parallel conversion (OpenMP first pass, `--threads`, tests, benchmark); faster legacy writer; lazy decoding
Samuel Garcin	Decode plan, C JIT with cache, LLVM ORC JIT, decode benchmark, differential tests
Mykhaylo Gershman	Self-describing format and converter, custom fields, validating reader and adapters, ATLAHS integration, runner architecture, test and benchmark suites, report

Questions?